

agent-Carl-Gustav-Jung · Agent 架构剖析与同类对标

它是怎么设计的、为什么这样设计、好处与代价 —— 兼一路走来的演进
2026-06-10

本报告剖析 agent-Carl-Gustav-Jung 的对话 Agent:重点不在"有哪些功能",而在"每一处为什么这样设计、买到了什么好处、付出了什么代价"。立场为「系统论述 + 诚实评估」——既讲清设计意图与独特性,也不讳言短板。所有技术细节均经源码逐行核实(正文附 文件:行 锚点),与同类 Agent 对标处给出 0-5 定性评分(非基准跑分,已注明)。面向自评、协作者与对外说明三类读者。

摘要

本系统不是一个纯 ReAct 的编码 Agent(逐步"想一步做一步、改文件跑 shell"),而是一个后台可恢复、记忆分层、支持群聊多人、可自我改进的对话 Agent。它的核心设计取向是把 Agent 的每一步落库为单一真相、用独立 worker 在 PostgreSQL 队列上可被另一进程接管续跑,并用一次规划 + 续跑重规划(plan-once + continuation-replan)而非纯逐步 ReAct 来换取成本、延迟与可恢复性。

双结论:立意与工程已落地且自洽——后台可恢复、双层记忆、控制面(中途改向 / 审批 / 暂停问人)、角色化子 Agent、MCP 双 transport、自我改进闭环均已实现并测试;代价也真实——对高度探索性的任务,plan-once 不如逐步 ReAct(故保留 M2 作为可选 flag);执行面刻意收窄(无 shell / 文件系统 / 浏览器);生态/通用子 Agent 不及 Claude Code 一类。

术语对照

中文	代码标识	含义
一次规划 + 续跑重规划	plan-once + continuation-replan	先生成整条 plan 再执行,plan 跑完且未达标时由反思决定是否带进展续跑(非逐步 ReAct)
运行 / 子运行	agent_run / parentRunId	一次 Agent 任务;子运行由 deep_research/s pawn_subagent 派生
步骤	agent_step	落库的最小单元(plan/tool_call/observe/reply/replan/approval_*/steer...),append-only
中途改向	steer	用户在运行中插话改变方向
审批门	approvalMode	工具级:auto 自动 / ask 需批准 / never 执行期硬拒
反思	reflection	收尾单点裁决"目标达成没、要不要续跑"
情景记忆	episodic / MAGI 层	外部 MAGI 服务的按需召回记忆(向量 + 时序失效)
核心记忆	memory_fragments	原生 always-on 稳定身份/偏好记忆
角色	AgentRole	子 Agent 的能力档:generalist/researcher/analyst
群聊协调	topicCoord	同 topic 多人触发时的 合并/排队/新建 决策
技能	topic_skill	注入 system prompt 的"群规";可由任务收尾自动蒸馏

Part A · Agent 是怎么设计的(核心)

本部分占全文重心。每个子系统按 设计 → 为什么这样(好处)→ 权衡/代价 三段展开。

A.1 总体设计哲学

四条贯穿全局的原则:

1. 单一真相落库:每一步(规划、工具调用、观察、回复、改向、审批)都 append-only 写入 agent_steps;运行级状态写 agent_runs。内存态从不作权威。
2. 后台可恢复:运行不绑定某个进程或某个 HTTP 连接;独立 worker 从 PG 队列捡活,崩溃后可被另一 worker 接管续跑。
3. fail-open(失败不致命):记忆写入、技能蒸馏、MCP 注册等"增益型"环节任何失败都不影响主任务收尾。
4. 持久化优于内存态:凡需跨步/跨续跑存活的状态(改向指令、被拒工具、检查点)一律落 run 级列,而非进程内变量。

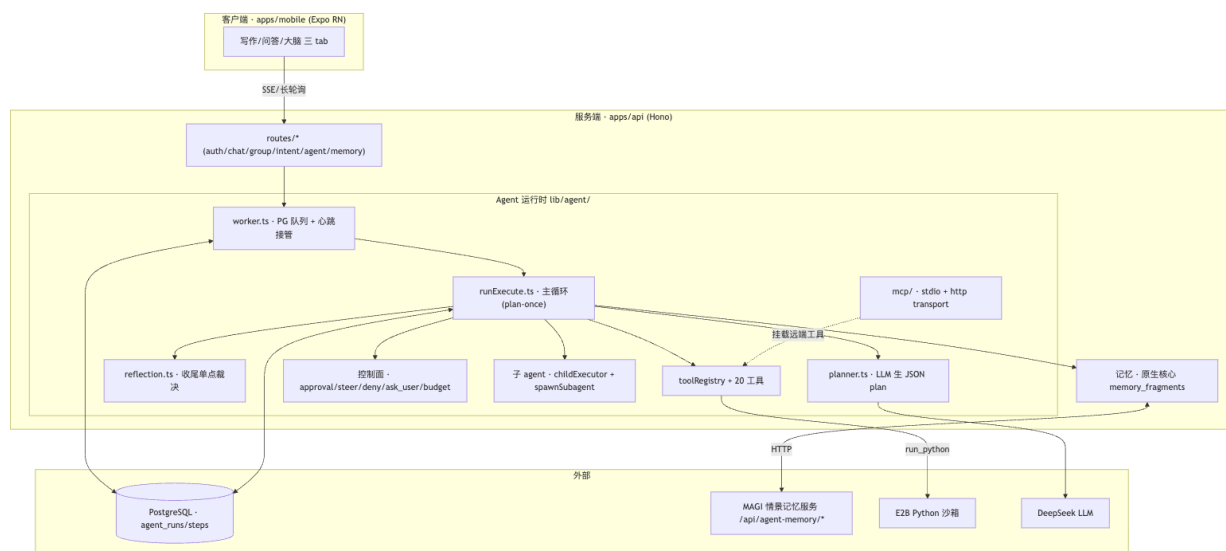


图 1

好处:这套"落库 + worker 队列"让 Agent 天然具备 *会话级 Agent 做不到* 的能力——关掉 App、换设备、服务重启,任务都还在跑且可续。代价:比"进程内一把梭"的会话 Agent 多了 DB 往返与状态机复杂度;但对一个要长期陪伴、跑长任务的助理型 Agent,这是值得的地基。

A.2 为什么是 plan-once + continuation-replan,而非纯 ReAct

设计:buildInitialPlan(runPlanGlue.ts)用一次 LLM 调用生成整条 JSON plan(planner.ts:generatePlanWithLlm,无 LLM/测试环境退化为 echo 桩);runExecute.ts 顺序执行 plan 的每一步;plan 跑完若 todo 未尽且预算未耗,由 reflection.ts 单点裁决是否带"已完成进展"续跑重规划。续跑硬上限 CONTINUATION_ROUND_CAP = 2(runExecute.ts:567)+ 无新进展的 stall guard。

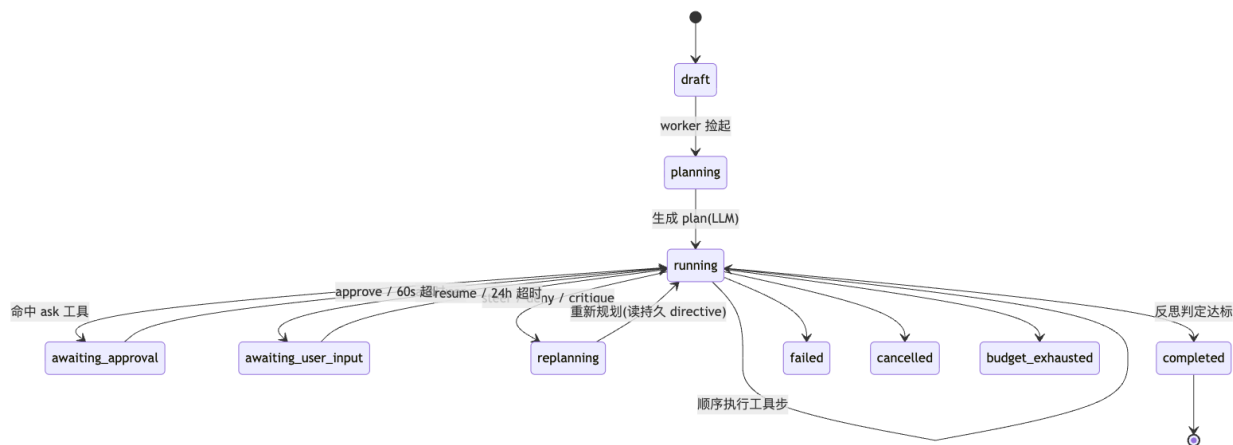


图 2

为什么这样(好处):① 省 token/延迟——一次规划胜过逐步多轮"想-做"往返;② 可后台可恢复——一条确定的 plan 落库后,worker 接管续跑天然成立(纯 ReAct 的"下一步全凭上一轮"难以干净恢复);③ 收尾有单点裁决——reflection 集中判断"达标没",避免逐步 ReAct 常见的"不知何时停"。

权衡/代价:对高度探索性任务(下一步强依赖上一步观察),一次规划不如逐步 ReAct 灵活。系统的补偿是 continuation-replan(带进展重规划)+ steer(用户改向),但终究不是纯 ReAct。这正是 M2 路线图保留"mode flag,plan_once↔react 共存"的原因——prototype 已测延迟,达标再 opt-in,不替换核心循环。

A.3 worker 队列与心跳接管 —— 最大差异化好处

设计:worker.ts 每 ~2s 一个 tick:用 pg_advisory_xact_lock 在事务内取 topic 槽位、pickupNextRun 捡 draft/running/replanning/queued 的运行、executeRun 异步派发(不 await);心跳戳 lastHeartbeatAt,崩溃后该 run 可被另一 worker 接管。每个 tick 还顺手:审批超时自动裁决、ask_user 24h 超时取消、群聊 ask_user 30s 升级全员可答。

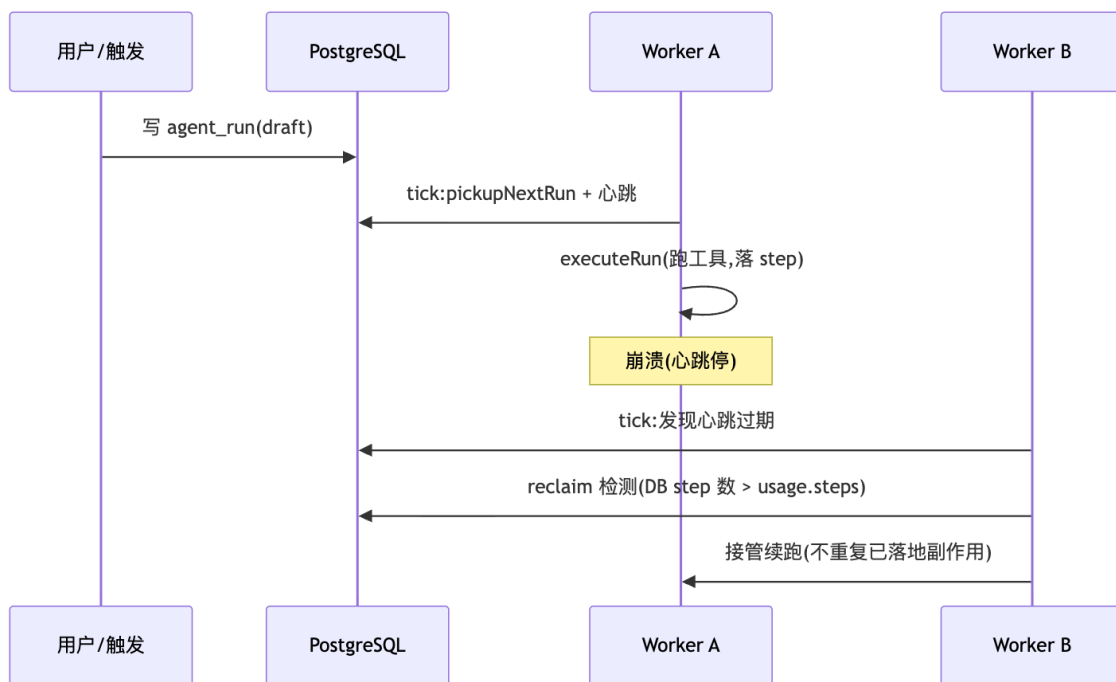


图 3

好处:崩溃/重启/扩容都不丢任务——这是把 Agent 从"会话玩具"变成"可托付的后台执行体"的关键。代价:需要幂等与 reclaim 配套(见 A.8),否则接管会重复副作用。

A.4 工具层:元数据驱动的安全与幂等

设计:toolRegistry 注册 20 个生产工具(registerAgentTools.ts)。每个 ToolDef 带元数据:
approvalMode(auto/ask/never)、costHint(low/medium/high)、hasSideEffects、idempotent、可选
computeIdempotencyKey。

工具	类别	approvalMode	幂等
search_papers / search_web / wikipedia / fetch_url / document_reader	只读检索	auto	
get_economic_series / magi_system_read / datetime_now	只读数据/知识库	auto	
youtube_transcript	只读检索	auto	
recall_memory / recall_step	记忆召回	auto	
critique_last_answer	元思自检	auto	
render_diagram	计算/出图	auto	
run_python	E2B 沙箱计算(副作用)	ask	
magi_content_ingest / doc_export_markdown	写入/导出	ask	
ask_user	暂停问人	auto	
deep_research / spawn_subagent	派子 agent	auto	

为什么这样(好处):① 审批分级——写入/高成本/沙箱工具默认 ask,只读检索 auto,无需逐工具写守卫;② 幂等键 + 唯一索引(agent_steps(run_id, tool_call_key).key 含 ownerId 名空间)——同一工具同参在一个 run 内恰好执行一次;命中缓存则写一条 observe 步留痕而不重复调用;③ 配合 reclaim,崩溃接管不会重复发邮件/重复写库这类副作用。

权衡:approvalMode 是工具级声明(并非独立的审批服务),never 走的是执行期硬守卫而非审批门——简单但意味着"审批策略"分散在各工具定义里。

A.5 控制面:中途改向与审批拒绝(本会话重点重构)

设计:运行进行中,用户可 steer(改向)、对 ask 工具 approve/deny、被 ask_user 暂停后 resume。本会话把 steer/deny 的重规划从早期"echo 占位桩"升级为真 LLM 重规划 + 持久化:

- steer:清 plan + 置 replanning + 把改向指令写入持久列 agent_runs.steer_directive(迁移 024);worker 接管时 buildInitialPlan 读它喂 LLM 重规划。
- deny:被拒工具名 append 进持久列 agent_runs.denied_tools(迁移 025);buildInitialPlan 每次都注入"不要调用 X";并在执行期加硬门(runExecute.ts 在 dispatch 前跳过被拒工具)。

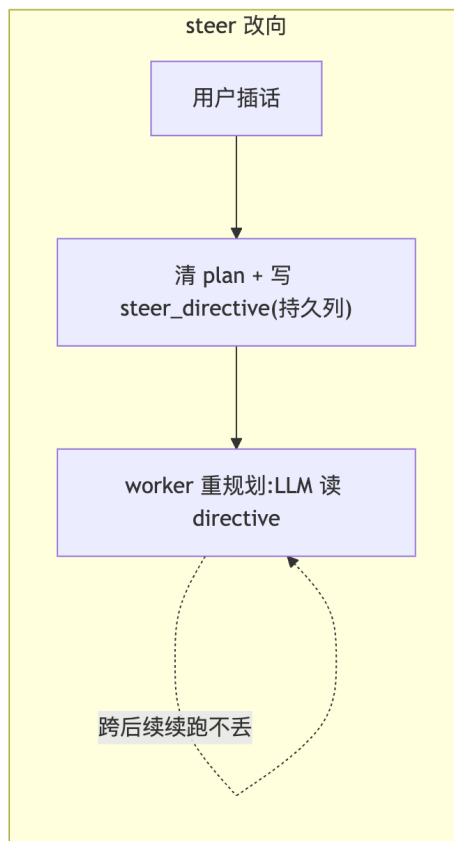
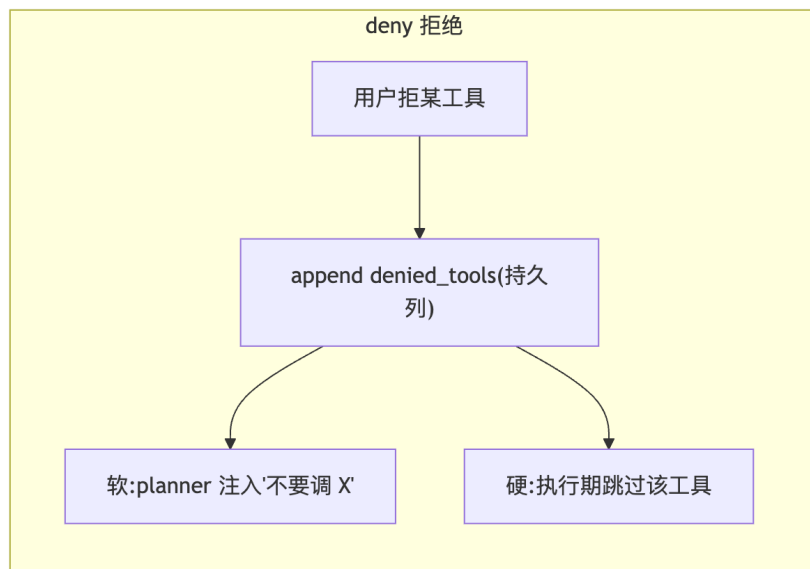


图 4

为什么这样(好处):早期 echo 占位桩的毛病是"接受了 steer 却不真改向 / deny 后工具又被重新规划"。持久列让改向/避坑跨多轮续跑不漂回原主题;软 prompt + 硬执行门双层确保即便 LLM 无视提示重拟被拒工具,执行期也兜底跳过、不会再撞审批门陷入循环。本会话以真模型活体验证:steer「改讲共时性」后终稿主题词频从持久化前的"共时性 1 / 个体化 5"变为"共时性 11 / 个体化 0"。

权衡:持久列目前只增不清(到 run 结束才失效)——对评审型场景刻意保持 focus-refresh 的新鲜度而非省调用(详见 Part C)。

A.6 子 Agent:角色化最小权限

设计(M3):deep_research 与 spawn_subagent 派生子运行(parentRunId),由独立 childExecutor 池跑。子 Agent 按 AgentRole(types.ts:3:generalist/researcher/analyst)获取工具子集 SUBAGENT_ROLE_TOOLS:researcher 是 9 个只读检索工具;analyst 额外加 run_python + render_diagram。

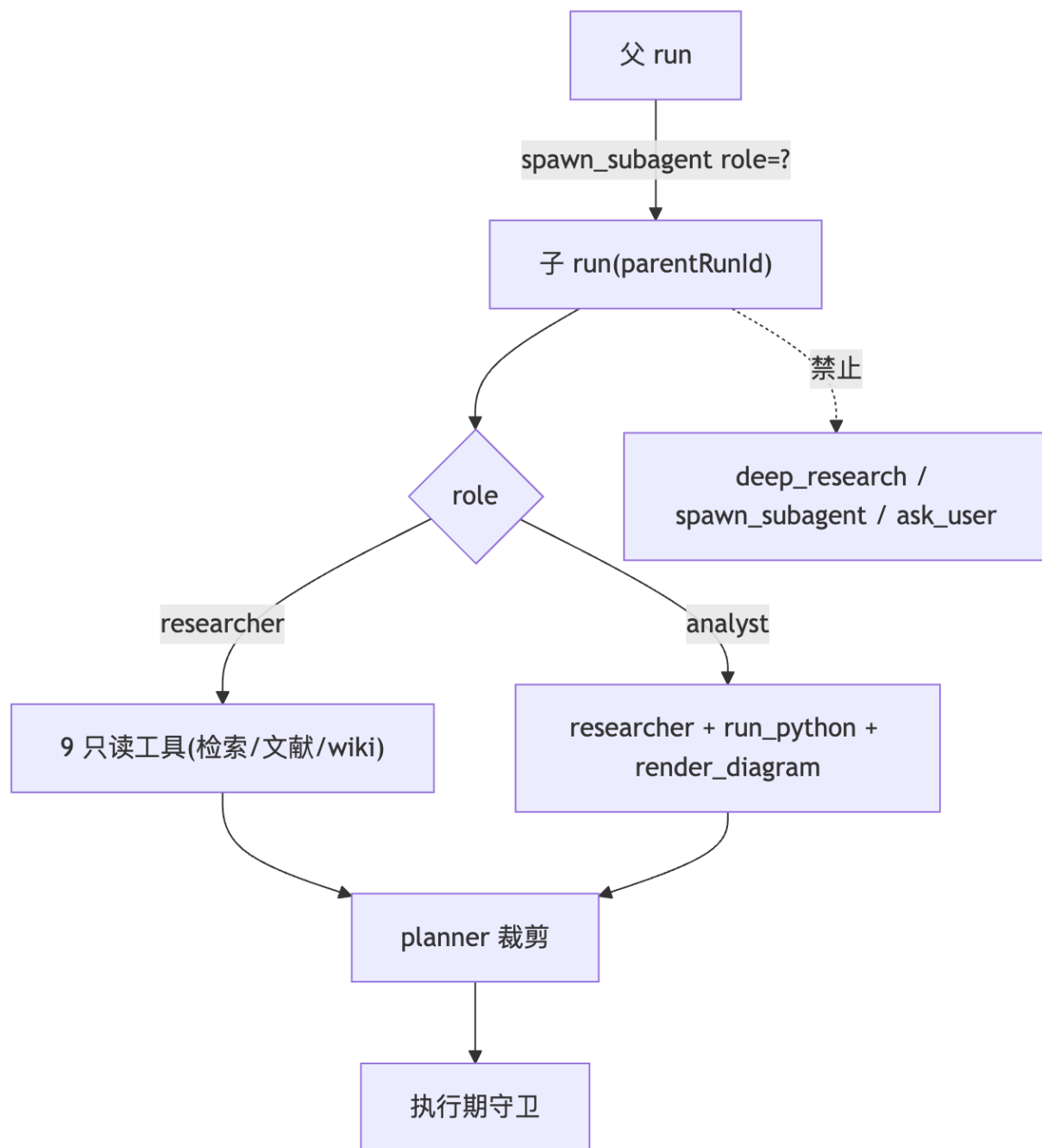


图 5

为什么这样(好处):① 最小权限——子 Agent 只拿任务所需工具,analyst 才碰沙箱 run_python,安全面收敛;② 双检——planner 裁剪工具列表 + 执行期再守卫一遍(防缓存/续跑 plan 绕过 planner);③ 防递归——任何角色子集都不含 deep_research/spawn_subagent/ask_user,子 Agent 无法再派子 Agent 或暂停。

权衡:deep_research 现等价于 spawn_subagent(researcher),两个工具对 planner 略有重叠(已记入 backlog,加第 4 个角色前重访)。

A.7 双层记忆:稳定核心 + 按需情景

设计:① 原生核心 memory_fragments——always-on 注入 system prompt 的稳定身份/偏好/项目记忆 (autoExtract 抽取、consolidate 合并、promote 升格;三 scope user/session/topic;confidence $\geq$ 0.85 才明

确"记住");② MAGI 情景层——外部 MAGI 服务(integrations/magi.ts → /api/agent-memory/*):bge 向量 + 稀疏混合检索、时序失效(invalidate)、质量门、主动召回 + recall_memory 工具按需召回。

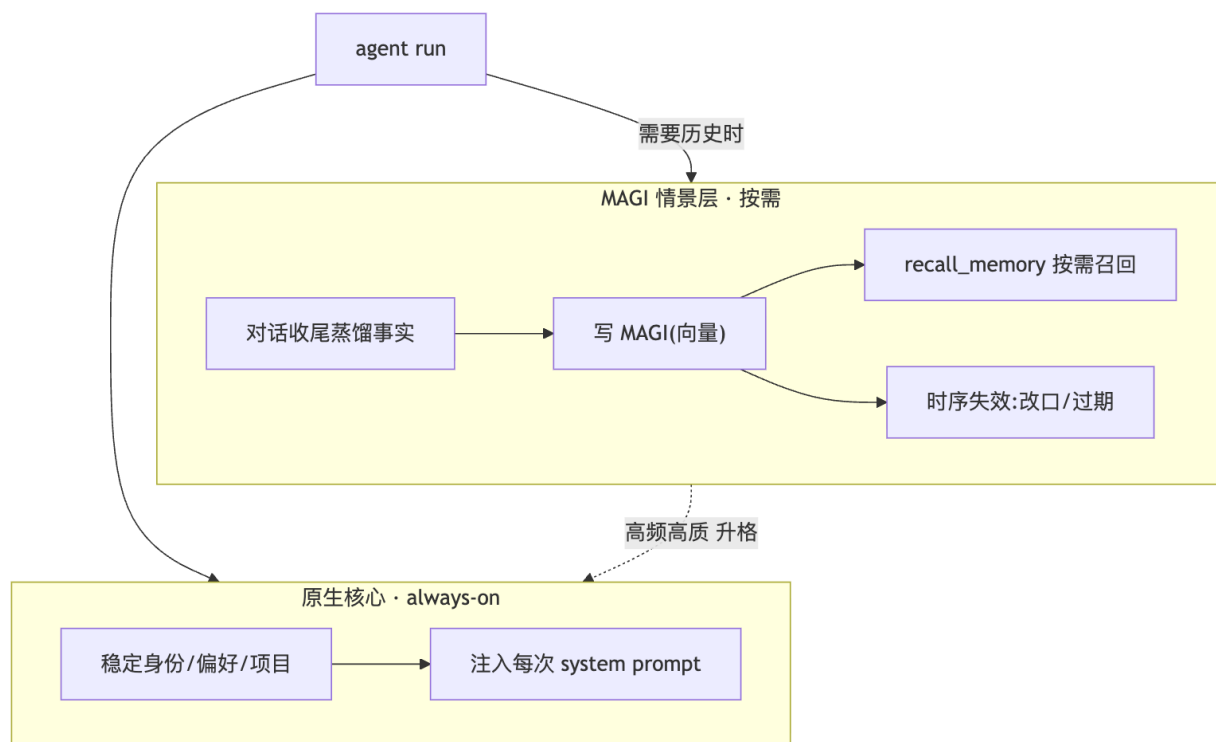


图 6

为什么这样(好处):稳定的"你是谁"常驻不耗检索;海量"聊过什么"按需召回不撑爆上下文;时序失效让"我改主意了"能覆盖旧事实;升格让反复出现的情景事实沉淀为核心。代价:依赖外部 MAGI 服务可用(未启用则 fail-open 返空);两层的一致性靠 owner 隔离 + 升格补偿维持。

A.8 韧性:分布式下副作用恰好一次

设计:幂等键(resolveToolCallKey,含 ownerId 名空间)+ agent_steps 唯一索引;命中则写 observe 缓存步;reclaim 检测(DB 已落 step 数 > usage.steps→补记 reclaim,接管不重复);E2B 沙箱按 run.sandboxId acquire/重连/失败新建,收尾 best-effort kill。

好处:多 worker 并发 + 崩溃接管下,带副作用的工具恰好一次;沙箱跨步持久(run_python 多次调用复用同一沙箱),终态回收。代价:沙箱超时回收后重连失败需新建,偶有冷启动延迟。

A.9 群聊协调:多人同 topic 不打架

设计(M7):topicCoord.ts 用 advisory lock 在事务内决策——子运行/私聊/无阻塞→新建;阻塞者同 owner 或 30s 窗口内(MERGE_WINDOW_MS=30_000)→合并(追问 append 进 mergedInputs);否则→排队。

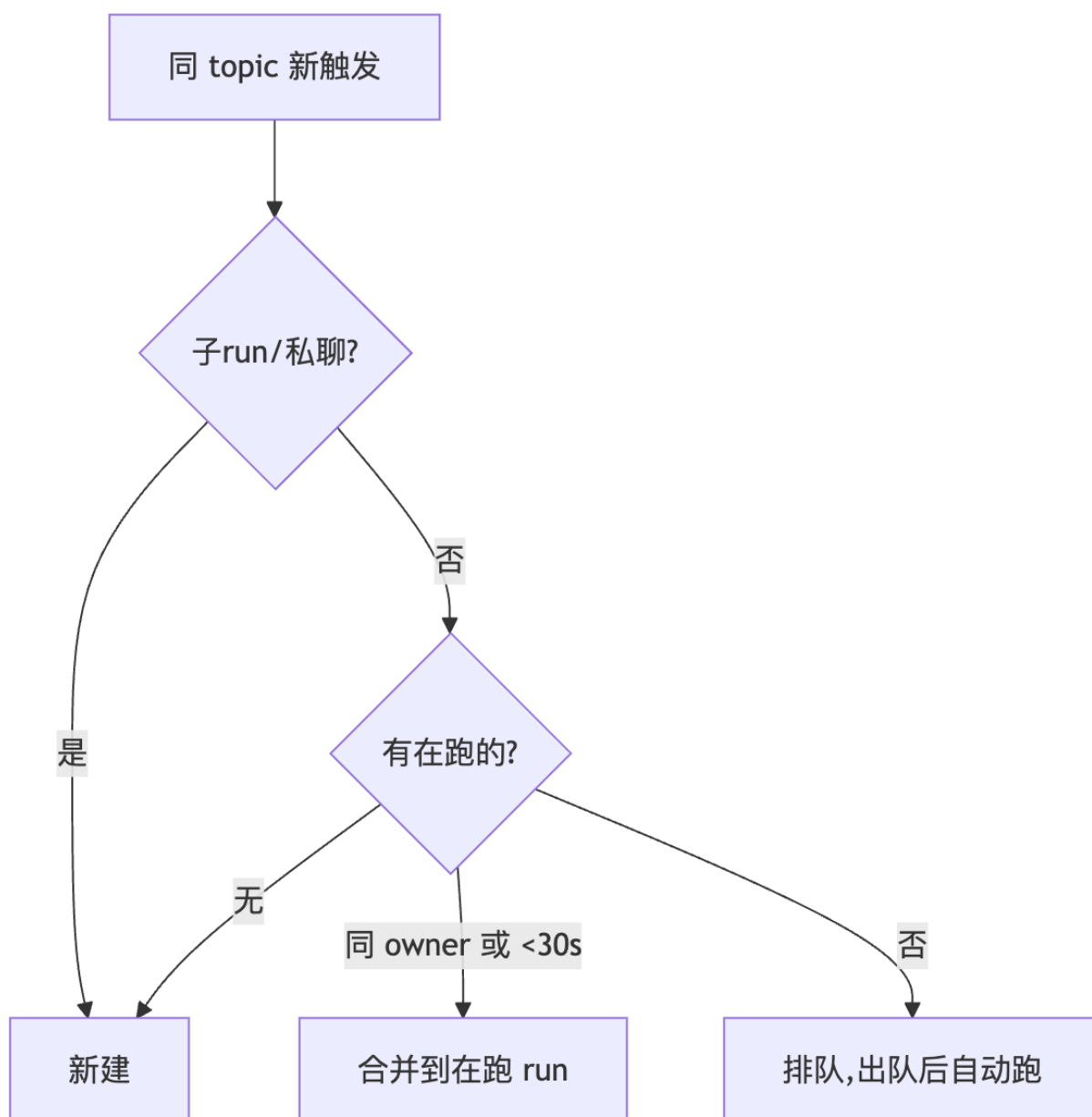


图 7

好处:群里多人几乎同时 @ 同一话题,不会起多个打架的 run,也不丢任何人的追问。代价:合并窗口是经验值(30s),跨人长间隔仍排队。

A.10 自我改进闭环

设计(M1/M5):成功的多步任务收尾时,自动把"这类任务怎么做"蒸馏成一条 topic_skill(source=auto_distilled、enabled=false 待评审);情景记忆可升格为核心;mobile 大脑 tab 提供技能/记忆/情景三个评审屏 + hub 待审 badge(本会话 #20)显示"N 条待审"。

好处:用得越久越懂你(技能/记忆沉淀),且人在环路——自蒸馏默认不启用,经用户评审才生效,防注入/污染。代价:需要用户去评审(故 #20 加 badge 提升发现性);技能注入有 prompt-injection 防御(high severity 拒、low warn)。

Part B · 与常见 Agent 对标

B.1 物种之辨

参照系两阵营:个人助理型(Hermes、OpenClaw:持久分层记忆 + 技能/自我改进 + soul + 常驻)与编码 Agent(Claude Code、Codex、Cursor:纯 ReAct + 文件/shell/代码工具 + plan mode)。本项目偏个人助理型,但叠加了编码 Agent 少见的后台可恢复 + 群聊多人。

B.2 述要

- Hermes / OpenClaw:常驻进程、分层记忆、技能模板、宽执行面(shell/文件/浏览器)。
- Claude Code / Codex / Cursor:会话级纯 ReAct,强在代码库内文件/shell/多文件编辑、通用子代理、MCP 客户端+服务端、用户可配 hooks。

B.3 能力对比

维度	本项目	Hermes	OpenClaw	Claude Code	Codex	Cursor
Agent 循环	plan-once+续跑(ReAct 留 flag)	ReAct	ReAct	纯 ReAct	纯 ReAct	纯 ReAct
后台/可恢复	PG 队列+心跳+幂等(强)	常驻	常驻	会话级	会话级	会话级
记忆分层	核心+情景两层(强)	5 层	SOUL+记忆	CLAUDE.md	AGENTS.md	规则/索引
自我改进/技能	自蒸馏+评审			skills	—	—
群聊多人	topic 合并/排队(罕见)	—	—	—	—	—
执行面宽度	web/文档/MAGI/沙箱 Python(无 shell/文件)	宽	宽	文件/shell	沙箱	代码库
通用子 agent	角色化(researcher/analyst)	—	—	通用	—	—
MCP	client(stdio+http)	—	插件	c/s	工具	工具
审批/沙箱	auto/ask/never+沙箱	—	权限			—

B.4 定位

在「后台可恢复 + 记忆分层 + 群聊多人」象限领先多数编码 Agent;落后处为「纯 ReAct 终态、执行面宽度(无 shell/文件/浏览器,刻意)、通用子 Agent 与生态」。

B.5 功能评分(0-5,定性,非基准)

维度(↑ 越好)	本项目	Hermes	Claude Code	Codex	Cursor
后台可恢复	5	4	1	1	1
记忆分层/长期	5	5	2	2	2
群聊多人协调	5	1	0	0	0
自我改进/技能	4	5	3	1	1
控制面(改向/审批/暂停)	4	2	4	4	3
子 agent 通用性	3	1	5	1	1
执行面宽度	2	5	5	4	4
编码/文件/shell	1	4	5	5	5

维度(↑越好)	本项目	Hermes	Claude Code	Codex	Cursor
上手/生态	2	3	5	4	5
MCP/可扩展	3	1	5	3	3

评析:本项目在后台可恢复 / 记忆分层 / 群聊上独占高分(竞品多为 0-1),在编码/文件/shell / 上手/生态上垫底——这是定位选择(对话陪伴型,非编码 Agent)的直接结果,非缺陷。

Part C · 现状、局限与逐设计 review

已落地:Part A 全部子系统 + M1(自蒸馏)/M3(子 agent)/M4(MCP)/M5(评审)/M7(群聊)+ 本会话 7 个已合 PR。
未竟:M2 ReAct 全量(仅 prototype)。

逐设计的诚实 review(代价):

- plan-once:省 token、可恢复,但探索性任务弱于逐步 ReAct → 靠续跑 + steer 补,终非纯 ReAct。
- 续跑 CONTINUATION_ROUND_CAP=2:防失控,但极复杂任务可能 2 轮不够 → 由 budget 兜底而非无限续。
- 持久列只增不清:steer_directive/denied_tools 注入到 run 结束:对评审 hub 刻意不加 TTL(freshness > 省调用:用户批准后返回必须看到更新计数)。
- 子 run 轮询:runChildSubagent 每 500ms 轮询 DB(至多 5min),未走 event-bus → 低负载无感,上规模是 DB 压力(backlog)。
- zenmux 占位:provider 字段有 zenmux,实际仅 DeepSeek 真实接。
- 执行面刻意收窄:无 shell/文件/浏览器——安全 + 聚焦,但也意味着不做代码 Agent 的活。

可证伪评测设计:① 记忆——A run 写事实、新 session B run recall_memory 跨会话召回 + 时序失效覆盖改口(本会话已活体验证);② 续跑——多步任务 plan 跑完未达标须续跑且 2 轮内收敛;③ 子 agent——researcher 子 run 试调 run_python 必被执行期守卫拒、analyst 放行(已单测);④ steer——改向后终稿主题词频应翻转(已活体验证)。

Part D · 路线图与"明确不做"

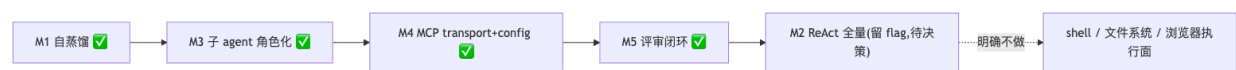


图 8

- 优先:M2 ReAct opt-in 共存(prototype 已测,需架构决策,专门推进)。
- backlog(低优):子 run 等待改 event-bus、count 端点(避免拉全列表计数)、deep_research↔spawn_subagent 重叠。
- 明确不做:shell / 文件系统 / 浏览器——刻意收窄执行面以保安全与聚焦。

Part E · 演进史("一路做了什么")

阶段	做了什么
基座	私聊/群聊对话 + 意图体系(agent_run 是被升格的一个意图)
运行时	worker PG 队列 + plan-once 主循环 + planner/reflection
韧性	续跑重规划(epic 0001-0003)+ 幂等 + reclaim + E2B 沙箱
记忆	原生核心 + MAGI 情景双层(向量/时序失效/升格)+ 主动召回
控制面	审批门 + ask_user + steer + budget
群聊(M7)	topic 合并/排队 + 技能三层注入 + 注入防御

阶段	做了什么
自我改进(M1/M5)	任务收尾自蒸馏技能 + episodic→core 升格 + mobile 评审屏/badge
子 agent(M3)	deep_research + 角色化 spawn_subagent + 工具子集守卫
MCP(M4)	stdio + Streamable HTTP/SSE transport + env 配置注册
本会话(2026-06-09~10)	steer/deny→M1c LLM 重规划 + 双向持久化 + deny 硬门(#16);M3-S1 角色子 agent(#17);MCP HTTP(#18)+ config(#19);M5 hub badge(#20)+ 渲染测试(#21);终态集合单一源(#22)

Part F · 反方与风险

诘问一:plan-once 是否够用? 对探索性任务确实弱。回应:故保留 M2 的 ReAct flag(prototype 已测延迟),并以 continuation-replan + steer 补位;若实测某类任务 plan-once 持续吃亏,即 opt-in ReAct——这是真实权衡,非已解。

诘问二:双层记忆会否噪声/泄漏? 海量情景若无组织即噪声;跨用户召回即隐私事故。回应:时序失效 + 质量门(conf 门)+ 升格控制信噪;owner 全程隔离(已活体验证换 owner 召回返空)。前提是 MAGI 服务可用,否则 fail-open 降级。

诘问三:子 agent 给 run_python 安全吗? analyst 子 Agent 可跑沙箱代码。回应:① 顶层本就能 run_python,委派给子 Agent 非提权;② E2B 沙箱隔离 + 子 run 预算(≤8 步/120s/50k);③ 角色双检 + 防递归。残余风险是沙箱本身的逃逸面,依赖 E2B。

结语 · 这样设计的好处(与代价)

把上面的设计选择合起来看,这套 Agent 的综合好处是:

- 1. 可托付——后台可恢复 + 幂等 + reclaim,让它能跑真实的长任务、扛崩溃重启,而不是一关页面就没。
- 2. 长期陪伴——双层记忆 + 自我改进闭环,用得越久越懂你,且人在环路防污染。
- 3. 安全可控——审批分级 + 角色最小权限 + 执行面刻意收窄 + 沙箱,把"Agent 乱来"的面收得很小。
- 4. 多人协作——群聊 topic 协调是多数 Agent 没有的能力。
- 5. 可干预——steer/deny 持久化让用户能真正改向、且改向不漂回。

换来的代价也清楚:不做编码 Agent 的活(无 shell/文件),plan-once 在探索性任务上不如纯 ReAct,依赖外部 MAGI/E2B/DeepSeek,状态机比"一把梭"复杂。

何时该选这套:要一个长期、可恢复、有记忆、能群聊、可审批的对话陪伴/研究助理。何时不该:要一个改代码、跑 shell、操作文件系统的编码 Agent——那是 Claude Code 一类的主场。

诚实声明:本报告所有架构论断均对应真实源码(apps/api/src/lib/agent/ 等,正文附 文件:行);评分为定性判断非基准跑分;局限与代价均如实列出,未夸大已完成度。